Unidad 2 / Escenario 4

Lectura fundamental

Arquitectura de transferencia de estado representacional

Contenido

1 Arquitectura de transferencia de estado representacional (REST)

2 Un ejercicio práctico para ver la diferencia entre REST y SOAP

Palabras clave: arquitectura, servicios, WSDL, cliente, protocolos.

1. Arquitectura de transferencia de estado representacional (REST)

Para describir REST primero se debe definir qué es un servicio web. Este es el medio o acción mediante la cual la web establece comunicación, es decir, habla con sus distintos elementos para garantizar su funcionamiento (envío de mensajes mediante un protocolo). En la lectura fundamental 3 se presentó la arquitectura orientada a servicios, y en ella se definió que un servicio web es aquel que permite interoperabilidad y extensibilidad es decir transmitir datos (protocolos y mensajes) "estructurados o no" usualmente a través de dicha arquitectura (Richardson & Ruby, 2007).

¿Sabía qué...?

Tres de los estándares web más conocidos y ampliamente utilizados son: HTML (HyperText Markup Language), para definir la estructura de los documentos; XML (eXtensible Markup Language), que sirve de base para un gran número de tecnologías; y CSS (Cascading Style Sheets), que permite asignar estilos para la representación de los documentos. Estos se encuentran dentro de una amplia lista de más de 120 tecnologías que, desde 1996, son administrados por la W3C, una organización que agrupa las opiniones y buenas prácticas de más de 400 empresas representativas de manera independiente y neutral (W3C, 2018).

1.1. ¿Para qué sirve un servicio?

Un servicio posibilita el diálogo entre aplicaciones empresariales y la hipermedia. Los mensajes contienen y hacen posible de cara al usuario la hipermedia, datos o información no lineal y dinámica variada: video, imagen, audio, coordenadas, mapas, texto y otro tipo de dato emergente con el que es posible interactuar. La hipermedia nace a partir de los conceptos de hipertexto y multimedia (García, 2018). Así, podemos decir que mezcla los dos conceptos: la multimedia para acuñar una forma de presentación y el hipertexto para hacer alusión a la funcionalidad de navegación conceptual.

Una arquitectura que permita ese paso ordenado de mensajes y el uso de un protocolo (REST o SOAP) hace posible el funcionamiento de la web o de sistemas empresariales y le da libertad al usuario o a los sistemas para navegar a través de la información o escoger el orden en que desea tener acceso a ella.

1.2. ¿Qué es REST?

La transferencia de estado representacional (REpresentational State Transfer) es un tipo de arquitectura de servicios orientado a la desconexión; es decir, que no es necesario un enlace permanente entre quien recibe y envía el mensaje. Un servicio REST no tiene estado (es stateless), lo cual significa que cada mensaje o uso del servicio debe ser autocontenido, ya que en cada interacción se pierden todos sus datos (Masoud & Mehta, 2013).

Las operaciones más importantes de REST y HTTP (Hypertext Transfer Protocol) son cuatro: POST (crear), GET (leer y consultar), PUT (editar) y DELETE (eliminar).

1.3. ¿Qué es SOAP?

SOAP es el protocolo basado en XML para arquitecturas orientada a servicios (SOA) (ver lectura fundamental 3). Este protocolo tiene dos versiones de SOAP en uso común: SOAP 1.1 y SOAP 1.2. (IBM®, 2018). Ambas están soportadas en múltiples tecnologías, como por ejemplo WebSphere Message Broker. SOAP está definido en los siguientes documentos emitidos por World Wide Web Consortium (W3C) (W3C, 2018):

- Simple Object Access Protocol (SOAP) 1.1
- SOAP Versión 1.2 Part 0: Primer
- SOAP Versión 1.2 Part 1: Messaging Framework
- SOAP Versión 1.2 Part 2: Adjuncts

Estos documentos emitidos por W3C explican en detalle la notación, codificación, estructura, uso, tipos de mensajes, entre otros aspectos del protocolo. Como ejemplo, en la figura 1, se presenta un mensaje de respuesta del protocolo SOAP.

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset="utf-8"
Content-Length: xxxx
<SOAP-ENV:Envelope
 xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
 SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"/>
 <SOAP-ENV:Body>
 <m:GetLastTradePriceResponse xmlns:m="Alguna-URI">
 <Price>52.6</Price>
 </m:GetLastTradePriceResponse>
 </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Figura 1. Estructura de código de un mensaje de respuesta con SOAP

Fuente: Elaboración propia (2018). Modificado de W3C (2018)

La primera línea contiene la especificación del protocolo y la línea 2 hace parte del encabezado del archivo que indica que se trata de un archivo XML con codificación de caracteres UTF-8, y la línea 3 señala la longitud del mensaje. El resto son la cabecera (SOAP-ENV:) y el mensaje respectivo (SOAP-ENV:Body.), que para el caso es la respuesta de la función (GetLastTradePriceResponse) que contiene el valor de algún artículo (<Price>52.6</Price>.).

1.4. Arquitectura SOA-REST

El diagrama arquitectónico de la figura 2, presenta el esquema de centralización de servicios mediante una arquitectura SOA de la aplicación Hola Doctor, que, de cara al usuario, recibe las peticiones e invoca a los servicios respectivos usando el patrón facade o fachada. La mensajería se realiza a través de los protocolos REST y SOA; nótese que los servicios bajo el protocolo SOAP requieren de un WSDL.

Cómo mejorar...

La mejor manera de familiarizarse con los protocolos es programando algunos de ellos y el mejor referente es el estándar oficial, que en la mayoría de los casos están dispuestos en la web y son de libre acceso como es el caso de W3C.

WSDL son las siglas de Web Services Description Language, un lenguaje de marcado extensible Extensible Markup Language (XML). En el WSDL se encuentra contenida toda la información del servicio. A través de este protocolo la aplicación Hola doctor se comunica con las otras aplicaciones o servicios (tarjeta de crédito, hospital y EPS).

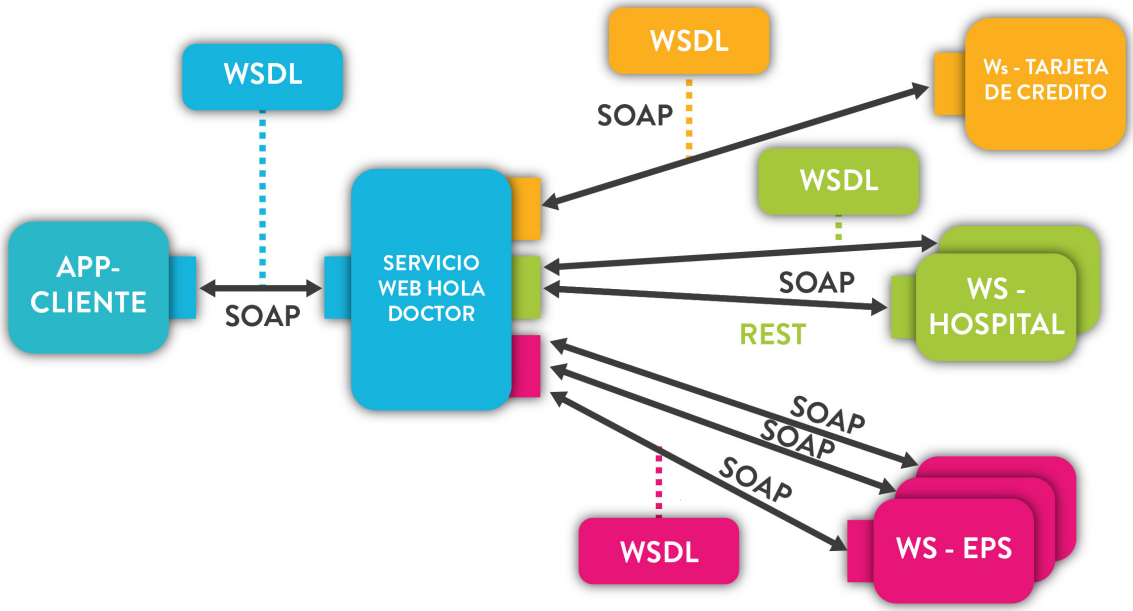

Figura 2. Arquitectura de servicios para una aplicación médica

Fuente: Elaboración propia (2018)

2. Un ejercicio práctico para ver la diferencia entre REST y SOAP

Si bien la finalidad de esta sección no es una práctica de programación de servicios web, el consumo o uso de estos servicios ayuda a el lector a entender mejor la dinámica de esta tecnología. La explicación teórica es fundamental; sin embargo, la implemnatación ayuda a entender mejor la dinámica. A continuación, se presenta, a manera de ejemplo, la forma como se puede utilizar un servicio web con SOAP y con REST.

2.1. Uso práctico de un servicio bajo protocolo SOAP

En este primer caso, se crea una aplicación Java con la cual se probará el servicio implementado; se puede buscar cualquier servicio en la web que esté bajo el protocolo SOAP. Para el ejemplo, se utiliza webservicex (ver figura 3).

Descripción del Servicio:
http://www.webservicex.net/ws/WSDetails.aspx?CATID=8&WSID=76
Contrato, wsdl:
http://www.webservicex.net/genericbarcode.asmx?WSDL

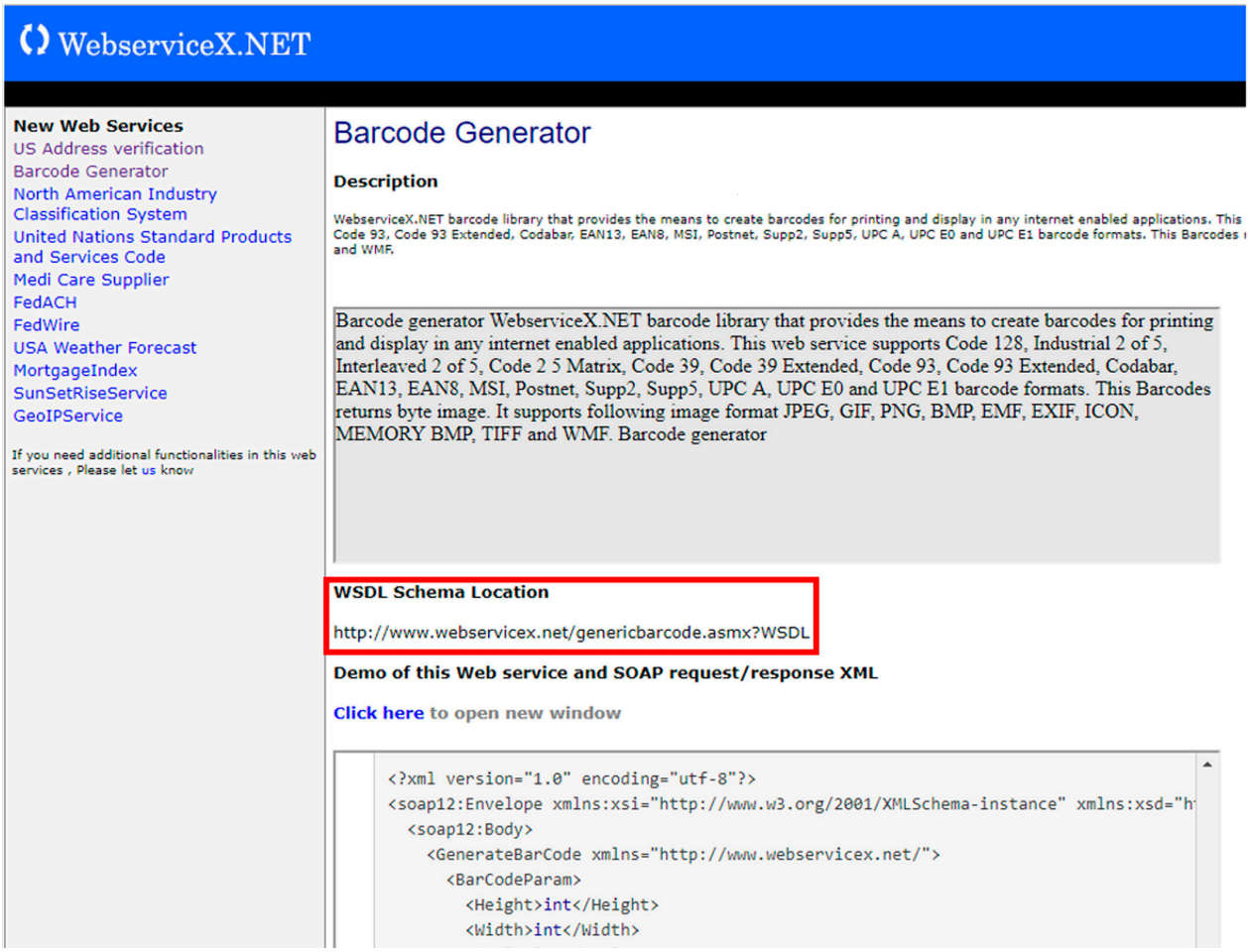

Figura 3. Servicio web bajo el protocolo SOAP

Fuente: elaboración propia

Luego creamos el cliente del servicio; desde el entorno de desarrollo integrado (IDE), de Netbeans, se crea una nueva aplicación Java, se selecciona la aplicación creada y se hace clic derecho, eso despliega una ventana de opciones; selecciona la opción new/Web Service Client (ver figura 4).

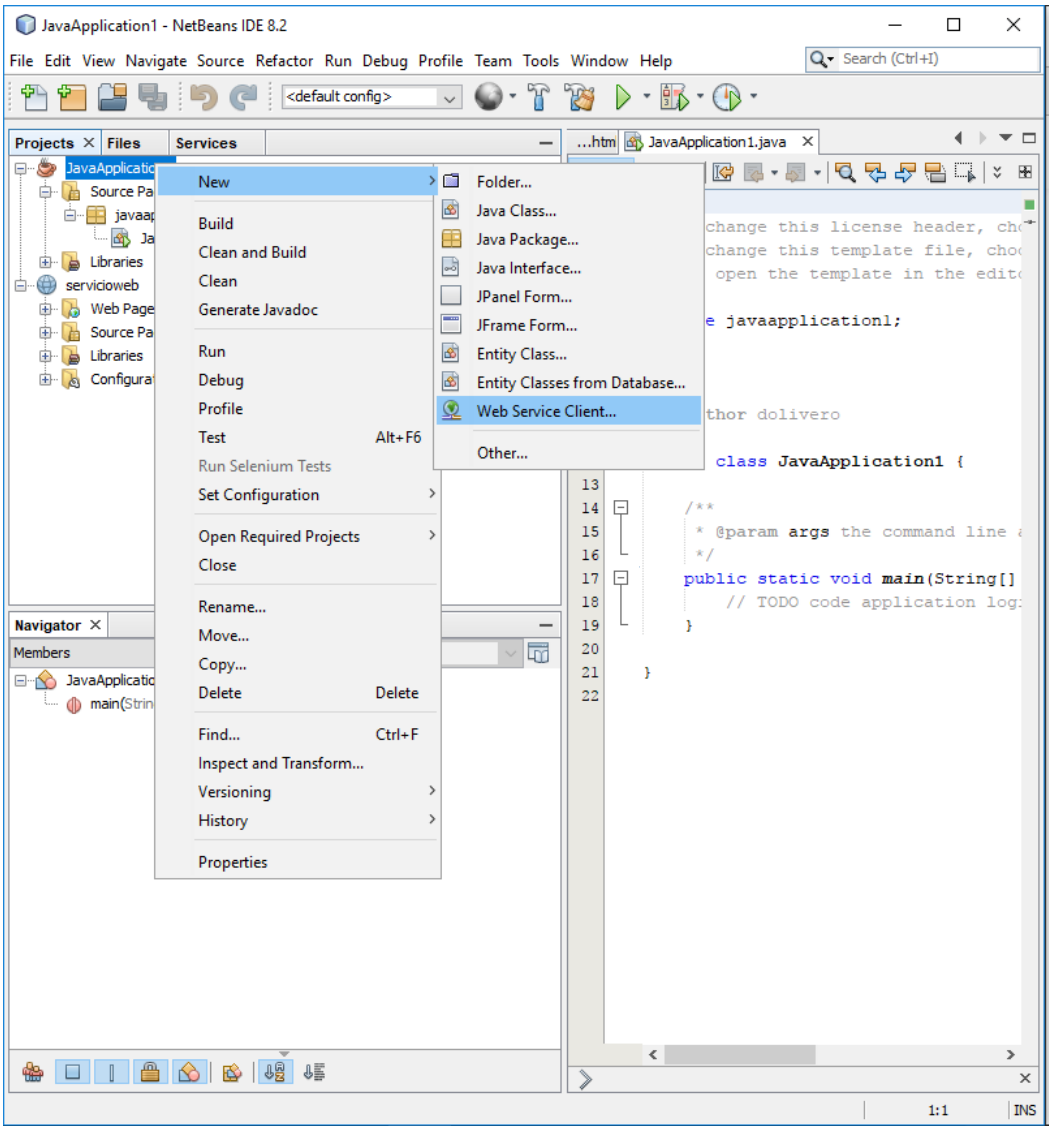

Figura 4. Creación de un cliente de servicio web a través de WSDL

Fuente: elaboración propia

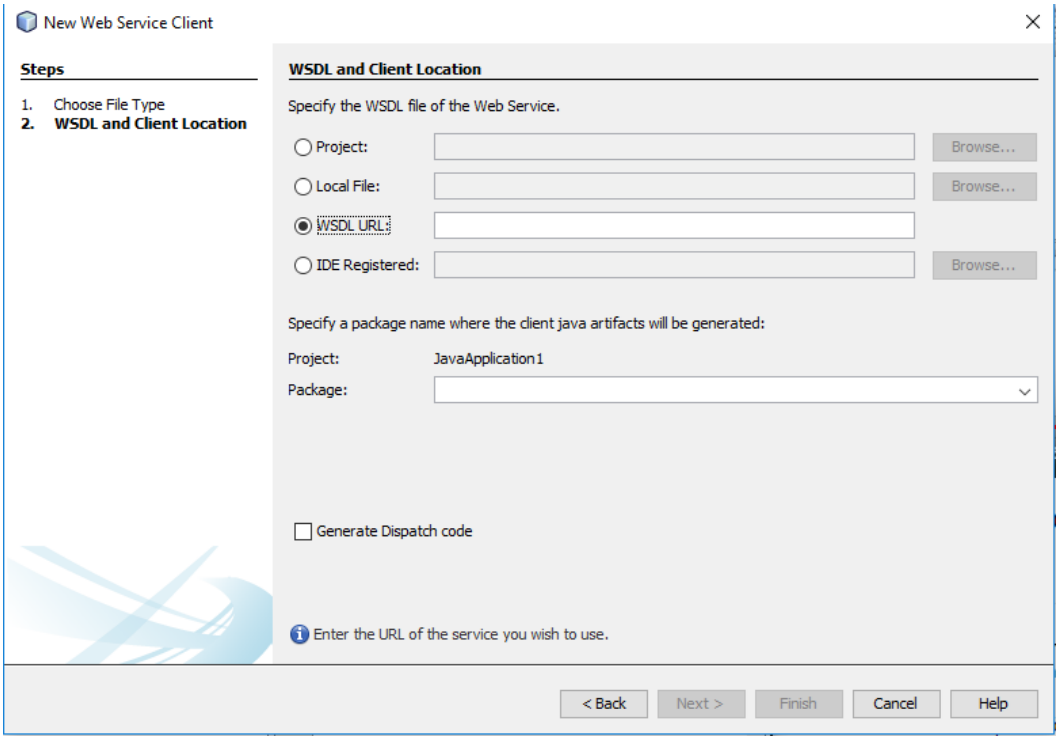

Figura 5. Creación de un nuevo cliente de servicio

Fuente: elaboración propia

Debemos colocar la URL del "contrato" o WSDL (XML de configuración del servicio) y la aplicación, automáticamente, a través del WSDL, traerá todos los métodos y especificaciónes del servicio. Para el ejemplo, seleccionamos la caperta del proyecto nominada web service references. En esta carpeta se encuentran todos los métodos; basta con seleccionarlo y arrastrarlo hacia la ventana de edición de la aplicación JavaAplication1.java, de esta manera se puede usar por invocación, en el método principal, a través de la función: getCitiesByCountry("Colombia") ver columna 2 de la figura 6.

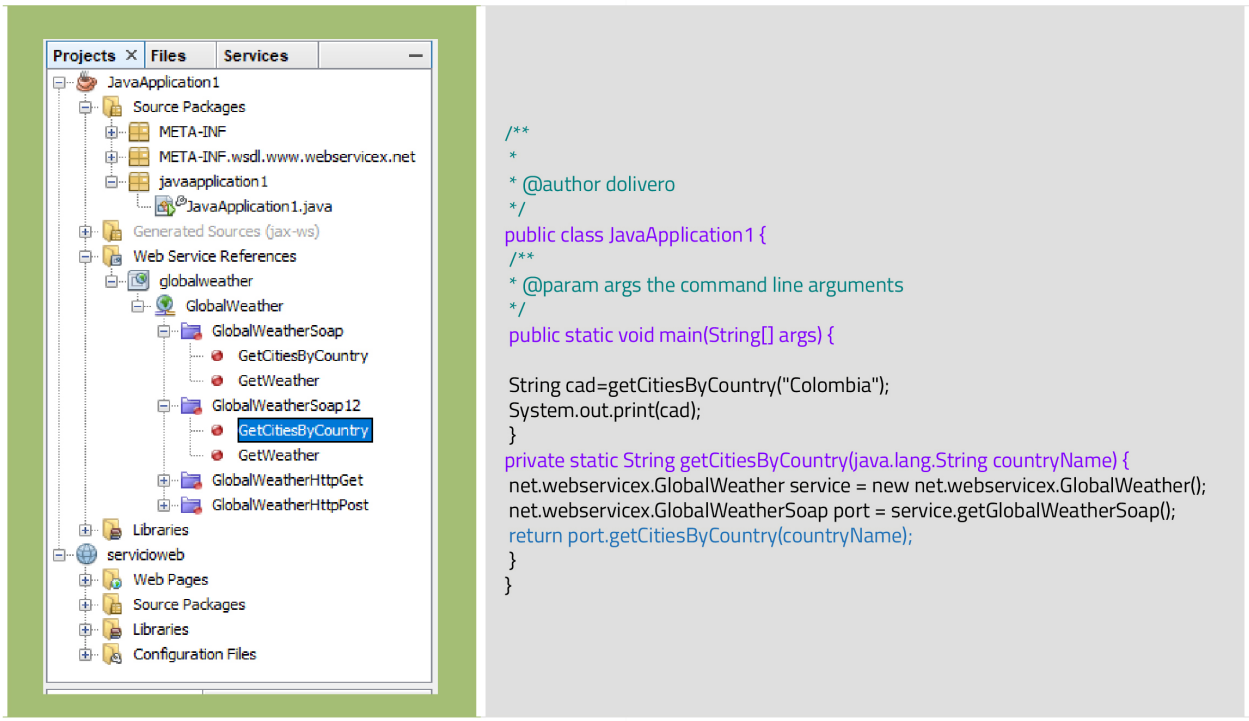

Figura 6. App de consumo de servicio SOA

Fuente: elaboración propia

En la ventana de ejecución podemos observar la salida del programa al probar el servicio mediante la función System.out.print(cad); (ver Tabla 1).

Tabla 1. Resultado de ejecución del aplicativo, datos del servicio web

Encabezado ventana de ejecución	ant -f C:\\Users\\dolivero\\Documents\\NetBeansProjects\\JavaApplication1 -Dnb.internal.action.name=run run init: Deleting: C:\Users\dolivero\Documents\NetBeansProjects\JavaApplication1\build\built-jar.properties deps-jar: Updating property file: C:\Users\dolivero\Documents\NetBeansProjects\JavaApplication1\build\built-jar.properties wsimport-init: wsimport-client-globalweather: files are up to date wsimport-client-generate: Compiling 1 source file to C:\Users\dolivero\Documents\NetBeansProjects\JavaApplication1\build\classes compile: run:

Salida del servicio	<NewDataSet> <Table> <Country>Colombia</Country> <City>Armenia / El Eden</City> </Table> <Table> <Country>Colombia</Country> <City>Puerto Asis</City> </Table> <Table> <Country>Colombia</Country> <City>Bucaramanga / Palonegro</City> </Table> <Table> <Country>Colombia</Country> <City>Bogota / Eldorado</City> </Table> <Table> <Country>Colombia</Country> <City>Barranquilla / Ernestocortissoz</City> </Table> <Table> <Country>Colombia</Country> <City>Buenaventura</City> </Table> <Table> <Country>Colombia</Country> <City>Cucuta / Camilo Daza</City> </Table> <Table> <Country>Colombia</Country> <City>Cartagena / Rafael Nunez</City> </Table> <Table> <Country>Colombia</Country> <City>Cali / Alfonso Bonillaaragon</City> </Table> <Table> <Country>Colombia</Country> <City>Barrancabermeja / Yariguies</City> </Table> <Table> <Country>Colombia</Country> <City>Ibague / Perales</City> </Table>	<Table> <Country>Colombia</Country> <City>Ipiales / San Luis</City> </Table> <Table> <Country>Colombia</Country> <City>Apartado / Los Cedros</City> </Table> <Table> <Country>Colombia</Country> <City>Leticia / Vasquez Cobo</City> </Table> <Table> <Country>Colombia</Country> <City>Medellin / Olaya Herrera</City> </Table> <Table> <Country>Colombia</Country> <City>Monteria / Los Garzones</City> </Table> <Table> <Country>Colombia</Country> <City>Mitu</City> </Table> <Table> <Country>Colombia</Country> <City>Neiva / Benito Salas</City> </Table> <Table> <Country>Colombia</Country> <City>Puerto Carreno / A. Guauquea</City> </Table> <Table> <Country>Colombia</Country> <City>Pereira / Matecana</City> </Table> <Table> <Country>Colombia</Country> <City>Popayan / Guillermo</City> </Table> <Table> <Country>Colombia</Country> <City>Pasto / Antonio Narin</City> </Table>	<Table> <Country>Colombia</Country> <City>Providencia Isla / El Embrujo</City> </Table> <Table> <Country>Colombia</Country> <City>Rionegro / J. M. Cordova</City> </Table> <Table> <Country>Colombia</Country> <City>Riohacha / Almirante Padilla</City> </Table> <Table> <Country>Colombia</Country> <City>San Jose Del Guaviare</City> </Table> <Table> <Country>Colombia</Country> <City>Santa Marta / Simon Bolivar</City> </Table> <Table> <Country>Colombia</Country> <City>San Andres Isla / Sesquicentenario</City> </Table> <Table> <Country>Colombia</Country> <City>Arauca / Santiago Perez</City> </Table> <Table> <Country>Colombia</Country> <City>Quibdo / El Carano</City> </Table> <Table> <Country>Colombia</Country> <City>Valledupar / Alfonso Lopez</City> </Table> <Table> <Country>Colombia</Country> <City>Villavicencio / Vanguardia</City> </Table> </NewDataSet>
Resultado y tiempo de ejecución	BUILD SUCCESSFUL (total time: 1 second)		

Fuente: elaboración propia

2.2. Uso práctico de un servicio bajo protocolo REST

En este ejercicio se toma como referencia un servicio aleatorio en Internet bajo el protocolo REST. Hay muchos servicios de consumo público en la web; para el caso, se presenta un servicio relacionado con la temperatura: Temperature Converter Web Service. Este servicio utiliza el método GET, mediante el cual debe enviar los paramentos junto con la URL; basta con agregar los parámetros al finalizar cadena o la URL para ver su funcionamiento; los paramentos del método GET exigidos son temperatura "temp" y escala "scale0". La cadena se conforma de la siguiente manera:

?temp=40&scale=F

El parámetro de temperatura es solo un número que representa la temperatura en Fahrenheit o Celsius. Por ejemplo, si se proporciona una temperatura Celsius de 40, el servicio web devolverá una cadena, como 104° F. Si se proporciona una temperatura Fahrenheit de 40, devolverá un Celsius de 4.44° C.

En la siguiente figura se observa, a la izquierda, el árbol de archivos del proyecto web creado y, a la derecha, el código del index.html con el código del formulario respectivo:

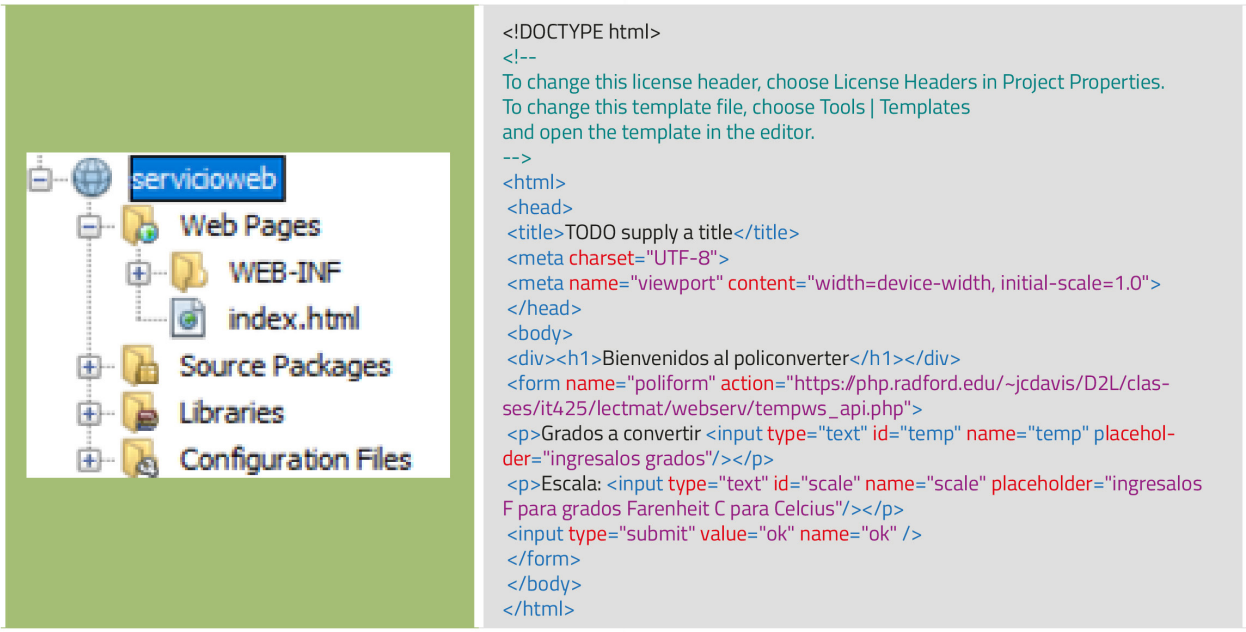

Figura 7. Consumo de servicio REST a través de formulario

Fuente: elaboración propia

Al ejecutar la aplicación web, se desplegará un formulario en el que se ingresan los dos valores (ver figura 8), al presionar el botón ok, ubicado en la parte inferior izquierda de dicha ilustración, se ejecuta la función submit, de acuerdo con la acción configurada en el formulario, y se envían todos los datos consignados en los campos de texto y capturados por el formulario (escala y grados) a través de la URL (ver tabla 2).

Figura 8. Formulario de conversor de escala

Fuente: elaboración propia

El resultado final se presenta en la figura 8 para ese caso, la cadena de texto de adición es la siguiente: **temp=3&scale=F&ok=ok**, que corresponde a la conversión de 3 grados Fahrenheit a centígrados. El resultado es -16.11° C.

Tabla 2. Respuesta del servicio web

URL	https://php.radford.edu/~jcdavis/D2L/classes/it425/lectmat/webserv/tempws_api.php**?temp=3&scale=F&ok=ok**
Resultado del servico	Es seguro | https://php.radford.edu/~jcdavis/D2L/classes/it425/lectmat/webserv/tempws_api.php?temp=3&scale=F&ok=ok -16.11° C

Fuente: elaboración propia

Un diagrama de secuencia que representa esta petición a la URL descrita anteriormente es:

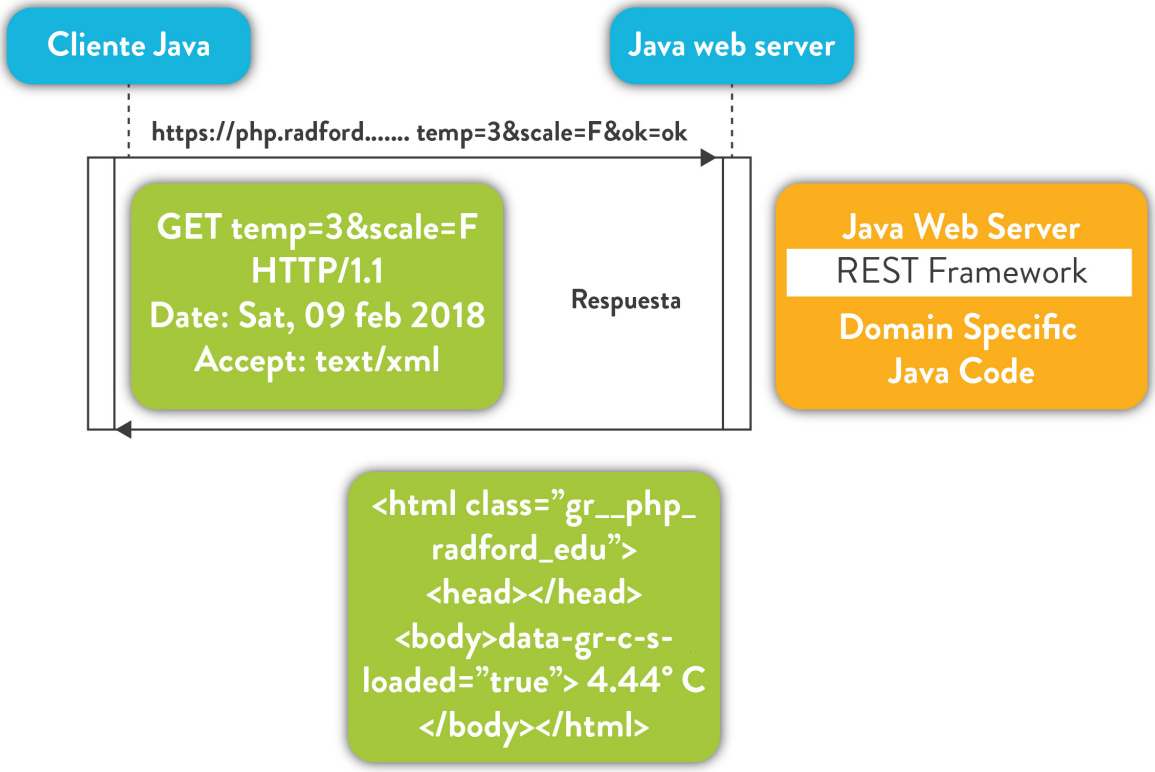

Figura 9. Diagrama de secuencia servicio REST

Fuente: elaboración propia

En síntesis...

SOA (Service-Oriented Architecture) es un tipo de arquitectura de *software* y no una tecnología o producto. Por otro lado, REST Y SOAP son protocolos o estándares. En el caso de SOAP, su comunicación y estructura es más compleja y requiere de WSDL para su interpretación, mientras que REST cuenta con una estructura liviana y puede transportar cualquier tipo de dato. Esto no significa que uno sea mejor que otro, simplemente tienen características que, según su aplicación, representan mayor o mejor ventaja.

Referencias

Alexander, C., Ishikawa, S. & Murray, S. (1980). *A Pattern Language Towns-Buildings-Construction.*

García, N. M. (9 de junio de 2014). *Definiendo Conceptos: Multimedia, Hipertexto e Hipermedia* [Mensaje de un blog]. Recuperado de http://e-ducacion.info/e-learning/definicion-multimedia-hipertexto-hipermedia/

IBM (28 de febrero de 2015). *WebSphere Message Broker, Versión 8.0.0.5 Sistemas operativos: AIX, HP-Itanium, Linux, Solaris, Windows, z/OS.* Recuperado de https://www.ibm.com/support/knowledgecenter/es/SSKM8N_8.0.0/com.ibm.etools.mft.doc/ac55770_.htm

Masoud, K. & Mehta, B. (2013). *Developing Restful Services with JAX-RS 2.0 WebSockets, and JSON.* Birmingham, U.K.: Packt Publishing Ltd.

Richardson, L. & Ruby, S. (2007). *RESTful Web Services*. New Jersey, CA. O'Reilly Media

Schmidt, D., Stal, M., Rohnert, H. & Buschmann, F. (2000). *Pattern-Oriented Software Architecture, Volume 2: Patterns for Concurrent and Networked Objects*. Chichester, England: Jhon Wiley & Sons.

Thomas, E. (2009). *SOA Design Patterns.* Boston: Prentice Hall.

Referencias de imágenes

W3C. (12 de 02 de 2018). *Simple Object Access Protocol (SOAP)*. Recuperado de https://www.w3.org/

INFORMACIÓN TÉCNICA

FACULTAD DE
INGENIERÍA, DISEÑO
E INNOVACIÓN

Módulo: Arquitectura del *software*

Unidad 2: Arquitectura del *software* orientada a servicios y REST.

Escenario 4: Arquitectura de transferencia de estado representacional

Autor: Diego Iván Oliveros Acosta

Asesor Pedagógico: Jeiner Velandia

Diseñador Gráfico: Alejandro Torres

Asistente: Leidy Alejandra Morales Eslava

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.